

OpenCode 五大核心机制

入门级实战教程 Cookbook

AGENTS.md · Agent · Skill · Command · MCP

定位 不是 OpenCode 全功能百科，而是一份专门教你把五个最容易混淆、又最重要的机制真正用起来入门教程。

版本基线：OpenCode 主线文档（非 v2 预览文档），资料核验日期 2026-08-13。

适合读者：已经知道 LLM / Coding Agent 是什么，但第一次系统配置 OpenCode；或者用过 OpenCode，却分不清 AGENTS.md、Agent、Skill、Command、MCP 的边界。

教学方式：全文使用一个虚构但可直接照搬的 TypeScript todo-api 项目作为统一例子。

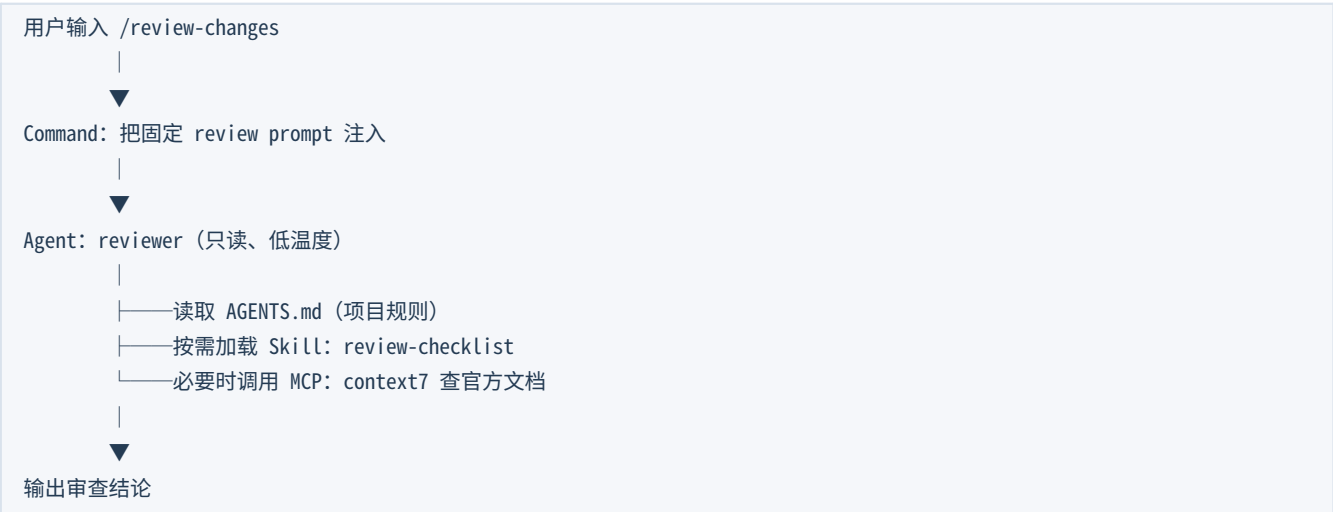
先看这 1 页：五者到底是什么？

如果只记住一张表，就记住这一张。

机制	一句话定义	主要解决什么问题	典型触发方式	是否常驻上下文
AGENTS.md	项目级长期规则/上下文	让 Agent 知道“这个仓库怎么做事”	OpenCode 启动/会话上下文	是（规则内容会进入模型上下文）
Agent	角色 + Prompt + 权限 + 模型	让不同工作由不同“角色”承担	Tab / @ / Task	该 Agent 工作时
Skill	可复用、按需加载的操作知识	把复杂 know-how 从常驻 Prompt 中移出去	模型调用 skill 工具	否，相关时再加载
Command	可重复执行的 Prompt 快捷入口	把高频人工指令产品化	/command	执行时注入
MCP	外部工具/系统接入协议	让 Agent 能操作外部能力	模型工具调用	工具定义会占 Context

核心关系 AGENTS.md 决定“共同规则”；Agent 决定“谁来做”；Skill 决定“按什么方法做”；Command 决定“怎么一键启动”；MCP 决定“能调用哪些外部能力”。

五者组合后的执行关系（概念图）



这份 PDF 的学习顺序

1. 先完成 15 分钟最小项目：只创建 AGENTS.md。
2. 再加一个只读 reviewer Agent，理解角色和权限边界。
3. 把 review 方法拆成 Skill，理解“按需加载”。
4. 把高频操作封装成 /review-changes Command。
5. 最后接 MCP，让 Agent 获得外部文档/系统能力。
6. 最终把五者组合成一套可维护的 OpenCode Starter Kit。

目录

建议第一次按顺序读；第二次把它当查阅手册。

- 01 准备：搭一个用于练习的 todo-api 项目
- 02 AGENTS.md：项目规则与长期上下文
- 03 Agent：角色、模型与权限边界
- 04 Skill：把 know-how 变成按需能力
- 05 Command：把高频 Prompt 变成可执行入口
- 06 MCP：给 Agent 接入外部工具
- 07 五者如何组合：完整 Starter Kit
- 08 排错：为什么我的配置没有生效？
- 09 设计原则：什么时候该用谁？
- 10 附录：可复制模板与学习路线

01

准备：先搭一个最小示例项目

所有后续示例都基于同一个 todo-api 仓库。

1.1 我们要得到什么

最终项目结构如下。你不需要一开始全部创建；每章会逐步加入。

```
todo-api/  
├── AGENTS.md  
├── opencode.jsonc  
├── package.json  
├── src/  
│   ├── index.ts  
│   ├── todo-service.ts  
│   └── todo-service.test.ts  
├── .opencode/  
│   ├── agents/  
│   │   └── reviewer.md  
│   ├── skills/  
│   │   ├── test-triage/  
│   │   │   └── SKILL.md  
│   │   └── review-checklist/  
│   │       └── SKILL.md  
│   └── commands/  
│       ├── review-changes.md  
│       └── explain-test.md
```

1.2 最小准备

进入项目并启动 OpenCode

```
mkdir todo-api  
cd todo-api  
git init  
opencode
```

如果还没有配置模型 Provider，在 TUI 中执行 `/connect`；选择可用 Provider 后再继续。

1.3 先不要急着做五种配置

入门最常见的错误，是第一天就同时堆十几个 Skill、五六个 Agent 和多个 MCP。这样一旦结果异常，你不知道到底是哪一层影响了模型。正确顺序是一次只加一层，并为每层设计一个可验证的小测试。

工程建议 配置增长顺序：AGENTS.md → Agent → Skill → Command → MCP。每加一层都先做“能发现、能触发、行为符合预期”的最小验证。

02

AGENTS.md：项目规则与长期上下文

先让 OpenCode “理解这个仓库”，再谈复杂 Agent。

2.1 AGENTS.md 是什么

OpenCode 官方将 AGENTS.md 作为自定义项目规则文件；内容会进入 LLM 上下文，用来针对项目定制行为。项目级 AGENTS.md 放在仓库根目录；全局规则可放在 ~/.config/opencode/AGENTS.md。

Know-why Agent 每次都能读源码，但“从源码推断项目做事方式”代价高且容易猜错。AGENTS.md 把高频、稳定、团队共享的隐性知识提前显式化。

2.2 最简单的创建方法：/init

```
cd todo-api
opencode
/init
```

官方说明中，/init 会扫描重要文件，并围绕未来 Agent 会话最需要的内容创建或改善 AGENTS.md，例如 build/lint/test 命令、执行顺序、架构、项目约定、setup quirks 与常见坑。如果文件已存在，它会尝试就地改善，而不是简单覆盖。

官方建议 项目级 AGENTS.md 应提交到 Git，因为它本质上是团队共享的 Agent 开发约定。

2.3 第一个可用 AGENTS.md

AGENTS.md

```
# todo-api

TypeScript REST API for a small todo application.

## Project structure
- `src/index.ts`: HTTP endpoint
- `src/todo-service.ts`: business logic
- `src/*.test.ts`: unit tests

## Commands
- Install: `npm install`
- Test: `npm test`
- Type check: `npm run typecheck`

## Working rules
- Make the smallest change that solves the task.
- Do not add dependencies unless necessary.
- Keep business logic out of the HTTP endpoint.
- After editing TypeScript, run the focused test first.
- Before finishing, run typecheck and inspect `git diff`.

## Safety
- Never run `git push` unless the user explicitly asks.
- Never modify production credentials or secrets.
```

2.4 为什么这份示例比“大而全项目百科”更好

内容	是否适合 AGENTS.md	原因
测试/构建命令	非常适合	高频且稳定，Agent 几乎每个编码任务都需要
仓库关键目录职责	适合	减少探索成本，避免放错代码
团队编码约定	适合	跨任务共享
一次性 bug 的全部调查过程	不适合	低复用，污染长期上下文
某个发布流程的 30 步细节	通常不适合	更适合做 Skill
“本次请检查第 38 行”	不适合	应留在当前 Prompt/Command

判断口诀 AGENTS.md 放“每次都可能用到、长期不太变、团队希望统一”的信息。一次性、低频、长流程知识应拆出去。

2.5 项目规则 vs 全局规则

位置	适合内容	是否共享
项目根目录/AGENTS.md	项目架构、命令、团队约定	通常提交 Git，团队共享
~/.config/opencode/AGENTS.md	个人偏好，例如输出格式	本机个人使用

OpenCode 还兼容 CLAUDE.md 作为 fallback：如果项目没有 AGENTS.md，可使用项目 CLAUDE.md；全局也有对应 fallback。对于新 OpenCode 项目，建议直接使用 AGENTS.md，减少认知分支。

2.6 规则优先级，入门必须知道

7. OpenCode 从当前目录向上寻找本地 AGENTS.md / CLAUDE.md。
8. 之后考虑全局 ~/.config/opencode/AGENTS.md。
9. 最后才是 ~/.claude/CLAUDE.md fallback（如果未禁用兼容）。
10. 同类位置中第一个匹配文件生效；项目同时存在 AGENTS.md 与 CLAUDE.md 时，以 AGENTS.md 为准。

2.7 AGENTS.md 太长怎么办：用 instructions 拆模块

OpenCode 支持在 opencode.json 中配置 instructions 数组，把已有规则文件一起加载。这非常适合团队已经有 CONTRIBUTING.md、测试规范或多包规则的情况。

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "instructions": [
    "CONTRIBUTING.md",
    "docs/guidelines.md",
    "packages/*/AGENTS.md"
  ]
}
```

Know-how 当规则“每都应该生效”时，用 instructions；当知识“只有某类任务才需要”时，更适合 Skill。两者不要混为一谈。

2.8 一个失败示例

```
# BAD AGENTS.md

Always be perfect.
Use best practices.
Write clean code.
Read all docs before every task.
Always run every test in the repository.
Never ask questions.
Use modern architecture.
```

问题不是这些话“绝对错误”，而是不可执行、无法验证、成本不可控。高质量规则应尽可能回答：什么时候做、做什么、如何验证、什么不能做。

2.9 把抽象规则改成可执行规则

模糊写法	更好的写法
Write clean code	Reuse existing abstractions before creating a new helper.
Run tests	After changing todo-service.ts, run its focused test first.
Be safe	Do not push, deploy, or change secrets without explicit request.

Understand architecture	Business logic belongs in service modules, not HTTP handlers.
-------------------------	---

练习：只用 AGENTS.md 约束一次修改

目标 体会“常驻项目规则”如何影响普通 Build Agent。

11. 创建上面的 AGENTS.md。
 12. 让 OpenCode：“给 todo-service.ts 增加 completeTodo 方法，并完成验证。”
 13. 观察它是否优先修改 service，而不是把业务逻辑写进 index.ts。
 14. 观察结束前是否执行 focused test / typecheck / git diff。
- 验收标准：Agent 的行为明显受项目规则约束；如果没有遵守，先改规则的可执行性，而不是马上创建新 Agent。

03

Agent：角色、模型与权限边界

不是多写一个 Prompt，而是定义“谁能做什么”。

3.1 Agent 的心智模型

```
Agent = Description
+ System Prompt
+ Model (可选)
+ Temperature / steps (可选)
+ Permission
+ Mode (primary / subagent)
```

OpenCode 的 Agent 是“专门化助手”。官方主线当前区分 primary agent 与 subagent：Primary 负责主对话；Subagent 承担局部专业任务，可由主 Agent 自动调用，也可通过 @ 手动调用。

3.2 先认识内置 Agent

Agent	类型	适合什么	典型权限特征
Build	primary	真正开发、改文件、跑命令	默认开发能力最完整
Plan	primary	分析、方案、审查	编辑与 bash 默认受限制/需审批
General	subagent	多步通用任务	较完整工具能力
Explore	subagent	快速搜索本地代码	只读
Scout	subagent	外部文档/依赖源码研究	只读，偏外部研究

入门建议 前一周先学会 Build / Plan / Explore 的分工，再创建自定义 Agent。很多任务根本不需要 10 个 Agent。

3.3 为什么要做自定义 Agent

创建自定义 Agent 的真正价值通常来自三个方面：稳定角色边界、稳定权限边界、稳定模型/推理参数。仅仅为了“换一个人格”通常收益不大。

3.4 创建第一个 reviewer Subagent

```
.opencode/
├── agents/
│   └── reviewer.md
```

`.opencode/agents/reviewer.md`

```
---
description: Review code changes for correctness, regressions, and missing tests
mode: subagent
temperature: 0.1
permission:
  edit: deny
  bash:
    "": ask
    "git diff": allow
    "git log*": allow
  webfetch: deny
---
```

You are a read-only code reviewer.

Focus on:

- correctness
- edge cases
- regression risk
- missing tests
- unnecessary complexity

Do not edit files.

Return findings ordered by severity.

3.5 怎么使用 reviewer

```
@reviewer review the current implementation
```

Markdown 文件名就是 Agent 名，因此 reviewer.md 对应 @reviewer。Subagent 也可能根据 description 被 primary agent 自动选中；description 越清晰，路由越可靠。

Know-why Agent 的 description 不是给人看的简介而已，它也是“什么时候该选这个 Agent”的路由信号。写成“Helpful assistant”几乎没有选择价值。

3.6 Permission 才是 Agent 的硬边界

OpenCode 当前主线 permission 有 allow / ask / deny 三种结果。新配置应优先使用 permission；官方 Permission 文档说明旧 tools 布尔配置自 v1.1.1 起已弃用，但仍为兼容性保留。

权限	含义	reviewer 中的建议
allow	无需批准执行	read / grep / git diff 等低风险只读动作
ask	执行前询问	少量不确定 bash
deny	直接阻止	edit、push、危险副作用

典型只读 Reviewer 权限

```
permission:
  edit: deny
  bash:
    "**": ask
    "git diff": allow
    "git log*": allow
```

关键细节 通配规则可能同时命中时，后匹配规则获胜。所以常见写法是先 `"**": ask`，再在后面列出允许的具体命令。

3.7 Primary 与 Subagent 如何选择

问题	Primary	Subagent
是否承载主对话	是	否，通常是子任务
适合长时间连续操作	是	可，但更适合边界清晰任务
是否适合隔离上下文	一般	非常适合
手动使用	Tab 切换	@agent
典型例子	Build / Plan	Review / Explore / Security audit

3.8 Context Isolation：Subagent 的隐藏价值

Subagent 不只是“多个模型并行”。更重要的是把大量中间搜索、尝试和局部推理放在子会话中，再把结果返回主会话，从而减少主上下文被噪声占满。

```
主 Agent
├─ @explore: 查 30 个文件 → 返回 8 行定位结论
├─ @reviewer: 检查 diff → 返回 5 个问题
└─ 主 Agent: 只保留真正需要决策的信息
```

3.9 Task Permission：限制 Agent 能调用谁

OpenCode 支持通过 `permission.task` 控制一个 Agent 可调用哪些 subagent。被 `deny` 的 subagent 会从 Task tool 描述中移除，从而减少模型把它当作候选动作。

```
{
  "agent": {
    "orchestrator": {
      "mode": "primary",
      "permission": {
        "task": {
          "**": "deny",
          "reviewer": "allow",
          "explore": "allow"
        }
      }
    }
  }
}
```

3.10 什么时候不该创建 Agent

- 只是一个可复用的 8 步流程，没有独立权限/模型需求：优先 Skill。
- 只是希望把同一句 Prompt 一键执行：优先 Command。
- 只是项目级共同规范：优先 AGENTS.md。
- 只是需要接 GitHub/Jira/数据库能力：优先 MCP/Tool。

练习：做一个只读 Reviewer

目标 理解 description、mode 与 permission 三个最重要字段。

15. 创建 .opencode/agents/reviewer.md。
16. 确保 edit: deny。
17. 在 Build 会话里对某个文件做一个小修改。
18. 使用 @reviewer review the current diff。
19. 尝试让 reviewer 直接修复问题，观察它是否被权限边界阻止。

验收标准：reviewer 能读和分析，但不能直接修改文件；它的输出聚焦在审查而非实现。

04

Skill: 把 know-how 变成按需能力

将“复杂流程知识”从常驻 Context 中拆出去。

4.1 Skill 是什么

OpenCode Agent Skills 通过 SKILL.md 定义可复用行为。关键点不是 “又一种 Prompt 文件”，而是按需加载：Agent 能看到可用 Skill 的名称与 description，真正需要时才通过原生 skill 工具加载完整内容。

Know-why 如果把发布流程、数据库迁移、故障排查、审查清单全写进 AGENTS.md，每次模型调用都要携带这些内容。Skill 把低频但重要的 know-how 变成 “可发现、需要时才加载” 的模块。

4.2 放在哪里

范围	推荐位置
项目级	.opencode/skills/<name>/SKILL.md
全局	~/.config/opencode/skills/<name>/SKILL.md
Claude 兼容	.claude/skills/<name>/SKILL.md 或 ~/.claude/skills/...
Agent Skills 兼容	.agents/skills/<name>/SKILL.md 或 ~/.agents/skills/...

4.3 命名与 frontmatter

主线官方文档要求 SKILL.md 以 YAML frontmatter 开始，name 与 description 必填；name 推荐/要求使用小写字母数字和单个连字符，并与目录名匹配。description 应具体到足以让 Agent 判断何时使用。

`.opencode/skills/test-triage/SKILL.md`

```

---
name: test-triage
description: Diagnose failing unit tests and propose the smallest safe fix
---

# Test triage workflow

Use this skill when one or more unit tests are failing.

## Procedure
1. Read the failing assertion and stack trace.
2. Reproduce the smallest failing test.
3. Identify whether the failure is code, test, fixture, or environment.
4. Locate the first incorrect assumption.
5. Propose the smallest fix.
6. Run the focused test again.
7. Run typecheck before finishing.

## Guardrails
- Do not weaken an assertion just to make a test pass.
- Do not delete tests without explicit justification.
- Prefer root-cause fixes over retries or sleeps.

## Output
Return:
- root cause
- changed files
- verification commands
- remaining uncertainty

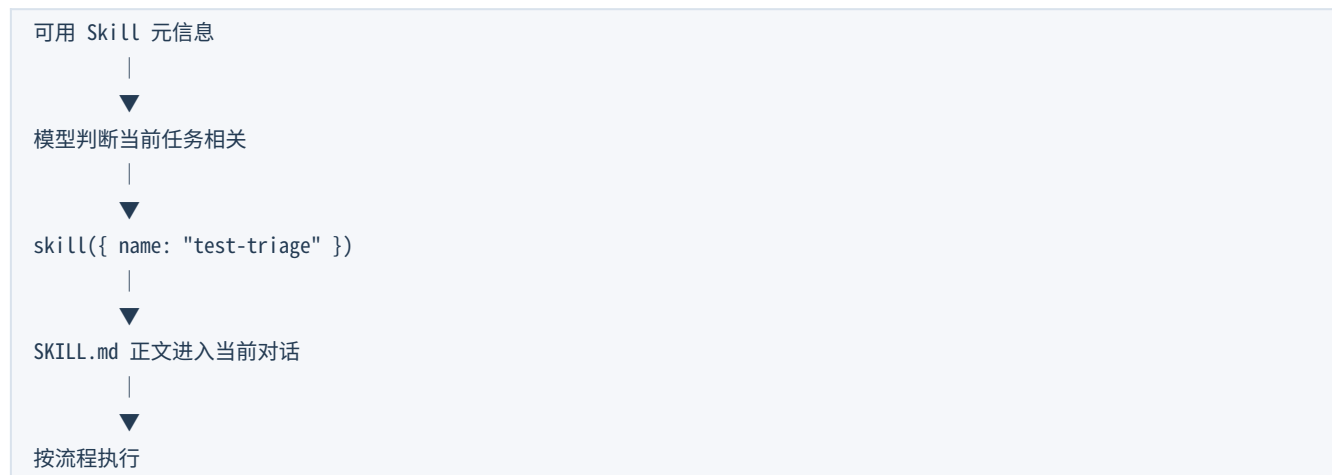
```

4.4 Skill 的 description 怎么写

差的 description	更好的 description	为什么
Helps with tests	Diagnose failing unit tests and propose the smallest safe fix	明确触发场景 + 任务目标
Review code	Review current code changes for correctness, regressions, and missing tests	与其他 Skill 更容易区分
Database stuff	Create and verify reversible schema migrations for this project	说明动作与约束

Know-how Skill description 是模型的“检索索引”。如果多个 Skill description 高度重叠，Agent 会更难稳定选中正确 Skill。

4.5 Agent 如何加载 Skill



也就是说，Skill 的价值不仅是复用，更是 Context Engineering：让大部分低频流程不占用每一步推理空间。

4.6 Skill Permission

opencode.json

```
{
  "permission": {
    "skill": {
      "": "allow",
      "internal-*": "deny",
      "experimental-*": "ask"
    }
  }
}
```

allow 表示立即加载；deny 会隐藏并拒绝访问；ask 会在加载前请求用户批准。你也可以为特定 Agent 单独覆盖 Skill 权限。

4.7 第二个 Skill：review-checklist

```
---
name: review-checklist
description: Review a git diff for correctness, edge cases, regressions, and missing tests
---

# Review checklist

## Read first
1. Understand the behavior the change intends to implement.
2. Read the changed code in surrounding context.
3. Check tests that should prove the behavior.

## Review dimensions
- Correctness: Does the implementation match the requirement?
- Edge cases: null/empty/error/concurrency/state transitions?
- Regression: What existing behavior may change accidentally?
- Tests: Is the important behavior actually asserted?
- Complexity: Is there a smaller change using existing abstractions?

## Reporting
List findings by severity.
For each finding include:
- file/location
- why it matters
- concrete failure scenario
- suggested direction

Do not edit files.
```

4.8 Skill vs AGENTS.md：最容易混淆的一组

内容	AGENTS.md	Skill
“测试命令是 npm test”	✓	通常不需要
“每次改 TS 都先跑 focused test”	✓	可作为流程补充
“排查失败测试的 7 步方法”	不建议	✓
“发布版本的完整 checklist”	不建议	✓
“业务逻辑放 src/service”	✓	不需要

4.9 Skill vs Agent：不要把方法论当角色

如果你想复用的是“怎么做 test triage”，那是 Skill；如果你需要一个“只能读测试、不允许改源码、使用便宜模型”的专门角色，那是 Agent。二者也可以组合：test-debugger Agent 在需要时加载 test-triage Skill。

test-debugger Agent

- └─ 权限: read/grep/bash test allowed, edit ask
- └─ 模型: 偏分析型
- └─ Skill: test-triage (按需加载)

练习：把方法论从 Prompt 拆成 Skill

目标 体验 Progressive Disclosure。

20. 创建 .opencode/skills/test-triage/SKILL.md。
21. 故意让一个简单单元测试失败。
22. 提示：“诊断这个失败测试，使用合适的 skill。”
23. 观察 Agent 是否选择 test-triage。
24. 把 description 改得很模糊，再观察选择是否变差。

验收标准：Skill 可被发现并加载；流程内容不需要长期写在 AGENTS.md。

05

Command：把高频 Prompt 变成可执行入口

把“我每次都要打同一句话”变成 /xxx。

5.1 Command 是什么

Custom Command 是一个可重复执行的 Prompt 模板。在 TUI 中输入 /command 即可运行。它是“人显式触发的工作流入口”，并不等同于 Agent 或 Skill。

5.2 第一个 Command：/review-changes

```
.opencode/  
├── commands/  
    └── review-changes.md
```

.opencode/commands/review-changes.md

```
---  
description: Review the current git diff without editing files  
agent: reviewer  
---  
  
Review the current changes.  
  
Current diff:  
!`git diff`  
  
Use the review-checklist skill if it is available.  
  
Return:  
1. critical issues  
2. medium-risk issues  
3. missing tests  
4. short overall assessment
```

文件名就是命令名，因此现在可以直接运行 /review-changes。因为 agent 指向 reviewer（subagent），它会默认以 subagent 调用方式执行。

5.3 Command 最重要的三种模板能力

参数：\$ARGUMENTS / \$1 / \$2

.opencode/commands/explain-test.md

```
---  
description: Explain a test failure  
---  
  
Explain why test `$1` is failing.  
Focus on file or module: $2  
Use the test-triage skill.
```

调用

```
/explain-test "should complete a todo" src/todo-service.test.ts
```

Shell 输出：!`command`

```
Recent changes:
!`git diff`

Recent commits:
!`git log --oneline -5`

Review these changes.
```

官方文档说明 command 中 shell 命令在项目根目录运行，输出会成为 Prompt 的一部分。

文件引用：@file

```
Review @src/todo-service.ts.
Compare its behavior with @src/todo-service.test.ts.
```

5.4 agent、subtask、model 三个高级选项

字段	作用	什么时候用
agent	指定谁执行	固定交给 reviewer / plan / build
subtask	强制以子任务方式运行	希望隔离主上下文
model	命令级覆盖模型	某个命令固定用快模型/强模型

```
---
description: Analyze a module in an isolated subtask
agent: plan
subtask: true
---

Analyze $ARGUMENTS.
Do not modify files.
```

Know-why Command 更像“入口函数”，而 Skill 更像“库函数/方法论”。Command 由人主动调用；Skill 可以由模型按任务语义自动选择。

5.5 Command vs Skill：一张表

问题	Command	Skill
谁决定触发	通常用户输入 /xxx	模型按 description 判断，也可显式要求
适合什么	高频固定入口	可复用流程知识
是否带参数	很适合 \$1 / \$ARGUMENTS	不是核心用途
是否适合注入 git diff	很适合 !`git diff`	一般不需要

是否可组合	可以调用某 Agent 并要求使用某 Skill	可被多个 Agent/Command 共用
-------	--------------------------	-----------------------

5.6 不要把 Command 写成巨型 Prompt

如果一个 Command 里塞入 300 行 review 方法论，后续每个类似 Command 都会复制粘贴。更好的方式是 Command 负责收集参数/上下文并指定目标，具体方法论放 Skill。

```
/review-changes
├─ 收集 git diff (Command)
├─ 交给 reviewer (Agent)
└─ review-checklist (Skill)
```

练习：做一个 /review-changes

目标 理解“入口”和“方法论”的分离。

25. 创建 review-checklist Skill。
 26. 创建 reviewer Agent。
 27. 创建 review-changes Command，并用 !`git diff` 注入差异。
 28. 执行 /review-changes。
 29. 确认输出来自 reviewer，且方法遵循 Skill，而不是在 Command 重复一套长清单。
- 验收标准：一个短 Command 能组合已有 Agent + Skill 完成完整工作流。

06

MCP：给 Agent 接入外部工具

MCP 解决的是“能力边界”，不是 Prompt 组织。

6.1 MCP 是什么

MCP (Model Context Protocol) 让 OpenCode 连接本地或远程 MCP Server，并把外部工具暴露给 LLM。加入后，这些工具会与 OpenCode 内置工具一起成为模型可调用的能力。

先记住 AGENTS.md / Skill / Command 主要在管理“信息与流程”；MCP 在扩展“Agent 能做什么”。

6.2 MCP 的成本：Context

OpenCode 官方特别提醒：MCP Server 会增加 Context 占用，工具很多时可能迅速变大；例如某些 GitHub MCP 会加入大量工具定义。入门不要把“能接的全部接上”，而应按任务启用。

6.3 Local MCP：最小测试

opencode.jsonc (官方文档用于演示的 test server)

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "mcp_everything": {
      "type": "local",
      "command": [
        "npx",
        "-y",
        "@modelcontextprotocol/server-everything"
      ]
    }
  }
}
```

Local MCP 常用字段包括 type=local、command；还可指定 cwd、environment、enabled、timeout。command 表示 OpenCode 如何启动这个本地 MCP 进程。

6.4 Remote MCP：以文档检索为例

opencode.jsonc

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp"
    }
  }
}
```

官方 MCP 页面当前用 Context7 作为远程 MCP 示例之一。若使用 API Key，可通过 headers + {env:...} 引用环境变量，不要把密钥直接写进仓库。

```
{
  "mcp": {
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp",
      "headers": {
        "CONTEXT7_API_KEY": "{env:CONTEXT7_API_KEY}"
      }
    }
  }
}
```

6.5 怎样告诉 Agent 什么时候用 MCP

一种方法是在 Prompt 里显式写 “use context7”；如果这是项目长期规则，也可以在 AGENTS.md 写：当需要查框架/依赖官方文档时，优先使用 context7。

AGENTS.md 中的 MCP 使用规则示例

```
## Documentation lookup
When implementation depends on external library behavior,
use `context7` tools to verify the current documentation
before guessing.
```

6.6 MCP Tool 的名字与权限

OpenCode 会给 MCP Server 的工具加 server name 前缀；官方文档示例说明可以用通配符匹配某个 MCP Server 的全部工具。Agent 文档也明确指出 permission key 可以按底层工具名做 wildcard，适用于 MCP Tool。

使用 permission 控制 MCP 工具（新配置优先 permission）

```
{
  "agent": {
    "reviewer": {
      "permission": {
        "context7_*": "allow",
        "other_mcp_*": "deny"
      }
    }
  }
}
```

版本注意 部分 MCP 官方示例仍展示 legacy tools: true/false 来控制可见性；OpenCode Permission 文档已说明通用 tools 布尔配置自 v1.1.1 起 deprecated。新教程优先使用 permission，旧写法只用于理解兼容配置。

6.7 OAuth MCP

远程 MCP 可以要求 OAuth。OpenCode 能自动发起认证流程；也可用 `opencode mcp auth <server-name>` 主动认证，用 `opencode mcp list` 查看状态，用 `opencode mcp logout <server-name>` 清理凭证。

```
opencode mcp list
opencode mcp auth sentry
opencode mcp logout sentry
```

6.8 MCP 的典型失败模式

症状	常见原因	第一步排查
工具完全不出现	MCP 未启动/配置未加载/被 deny	<code>opencode mcp list</code> + 检查配置位置
Local MCP 启动失败	<code>command</code> / <code>cwd</code> / 环境变量错误	在终端手动跑 <code>command</code>
Remote MCP 401	OAuth/API Key 未配置	<code>auth</code> / <code>headers</code> / <code>env</code>
模型几乎不用它	工具太多或 Prompt/规则不明确	缩小 MCP + 明确适用条件
Context 快速膨胀	Server 暴露工具太多	只启用需要的 MCP/按 Agent 限制

6.9 MCP vs Skill

场景	应该用
“怎么做一次安全数据库迁移”	Skill
“真正查询数据库 schema”	MCP / Tool
“怎么进行 PR Review”	Skill
“读取 GitHub PR / issue”	MCP
“什么时候应该查外部文档”	AGENTS.md 规则
“一键开始查文档并生成报告”	Command，可组合 MCP

练习：接入一个最小 MCP

目标 理解“能力”与“流程”的区别。

- 30. 先选择一个官方文档中的测试/文档检索 MCP 示例。
- 31. 写入 `opencode.jsonc`。
- 32. 重启/刷新 OpenCode 后查看 MCP 状态。
- 33. 用明确 Prompt 要求使用该 MCP 完成一次只读查询。
- 34. 再给 reviewer Agent 加 permission，只允许需要的 MCP 工具。

验收标准：MCP 工具能被发现并调用；Agent 权限能限制其调用范围；不需要把 MCP 的工具说明复制到 Skill。

07

五者如何组合：完整 Starter Kit

把“规则、角色、方法、入口、能力”串成一个小 Agent 系统。

7.1 完整结构

```
todo-api/  
├── AGENTS.md  
├── opencode.jsonc  
├── .opencode/  
│   ├── agents/  
│   │   └── reviewer.md  
│   ├── skills/  
│   │   ├── test-triage/SKILL.md  
│   │   └── review-checklist/SKILL.md  
│   └── commands/  
│       ├── review-changes.md  
│       └── investigate-test.md
```

7.2 AGENTS.md：只留共同规则

```
# todo-api  
  
## Structure  
- `src/index.ts`: HTTP boundary  
- `src/todo-service.ts`: business logic  
- `src/*.test.ts`: unit tests  
  
## Verification  
- Focused test first.  
- Then `npm run typecheck`.  
- Inspect `git diff` before finishing.  
  
## Architecture  
- Keep business logic out of HTTP handlers.  
- Reuse existing service abstractions.  
  
## External docs  
- If library behavior is uncertain, verify it with `context7` rather than guessing.  
  
## Safety  
- Do not push or deploy without explicit user request.
```

7.3 reviewer Agent：只负责审查

```
---
description: Read-only reviewer for correctness, regressions, and missing tests
mode: subagent
temperature: 0.1
permission:
  edit: deny
  bash:
    "**": ask
    "git diff": allow
    "git log*": allow
  skill:
    "review-checklist": allow
    "**": ask
  context7_*: allow
---
```

Review changes without editing files.
Prefer evidence from code, tests, and project rules.
Use external docs only when library behavior is uncertain.

7.4 Skill：定义“怎么审查”

```
---
name: review-checklist
description: Review a git diff for correctness, edge cases, regressions, and missing tests
---
```

1. Understand intended behavior.
2. Read changed code in context.
3. Check edge cases and state transitions.
4. Check regression risks.
5. Check whether tests prove the new behavior.
6. Report findings by severity with concrete failure scenarios.

7.5 Command：定义“一键怎么启动”

```
---
description: Review current git changes with the read-only reviewer
agent: reviewer
---
```

Current diff:
!`git diff`

Review these changes.
Use the review-checklist skill.

7.6 MCP：只提供外部能力

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp"
    }
  }
}
```

7.7 现在执行 /review-changes 时发生什么

35. OpenCode 会话已经带着项目 AGENTS.md 的共同规则。
36. 用户输入 /review-changes。
37. Command 运行 git diff，把结果注入 Prompt，并指定 reviewer。
38. reviewer 以 subagent 身份工作，edit 被 deny。
39. reviewer 根据任务加载 review-checklist Skill。
40. 如果遇到依赖库行为不确定，可调用被允许的 context7 MCP 工具。
41. reviewer 返回审查发现，主上下文不用保留大量中间探索细节。

真正的设计思想 五个机制不是平级的“配置文件集合”，而是五个不同维度：Context、Role、Procedure、Entry Point、Capability。好的设计让每种信息只放在最适合的一层。

7.8 再做一个“失败测试排查”组合

```
/investigate-test <test name>
|
├─ Command: 收集参数
├─ Agent: Plan 或 test-debugger
├─ Skill: test-triage
├─ AGENTS.md: 项目测试命令/目录规则
└─ MCP: 通常不需要；只有依赖行为不确定时再查
```

注意：不是每个工作流都必须把五者全部用上。能用 2 层解决，就不要强行用 5 层。

08

排错：为什么我的配置没有生效？

先确认“发现”，再确认“触发”，最后确认“权限”。

8.1 通用排错三层法

Layer 1 Discovery: OpenCode 找到文件了吗?

Layer 2 Routing: 模型/用户触发正确对象了吗?

Layer 3 Permission: 对象有权执行需要的动作吗?

8.2 AGENTS.md 不生效

- 确认 OpenCode 是从预期目录启动的。
- 确认项目根目录/父目录是否存在另一个优先级更高的 AGENTS.md。
- 如果同时有 CLAUDE.md，记住 AGENTS.md 优先。
- 规则是否过于抽象，导致无法验证？把“be clean”改成具体动作。
- 规则是否应该拆到 instructions 或 Skill，而不是让 AGENTS.md 无限增长？

8.3 Agent 不出现 / 不被自动选中

- 文件是否位于 .opencode/agents/ 或全局 agents 目录。
- 文件名是否就是你 @ 使用的 Agent 名。
- mode 是否允许作为 subagent/primary 使用。
- description 是否清晰到能与其他 Agent 区分。
- Task permission 是否把它 deny 掉；注意用户手动 @ 与 Agent 自动 Task 调用是两回事。

8.4 Skill 不出现

- 目录形式是否为 .opencode/skills/<name>/SKILL.md。
- SKILL.md 大小写是否正确。
- frontmatter 是否包含 name 和 description。
- name 是否与目录匹配，并满足小写 kebab-case。
- permission.skill 是否 deny 了它。
- 是否存在同名 Skill 导致来源冲突。

8.5 Command 不出现 / 参数不对

- 文件是否在 .opencode/commands/。
- 命令名来自 markdown 文件名。
- 需要整段参数用 \$ARGUMENTS；需要位置参数用 \$1、\$2。
- 命令里 !`...` 会真实运行 shell，先确认命令安全且输出大小可控。
- 自定义 Command 可覆盖内置同名命令，避免无意使用 /init、/help 等名称。

8.6 MCP 不工作

- 先用 opencode mcp list 看 Server 是否配置和连接成功。
- Local: 手动运行 command，看依赖/环境变量/cwd 是否正常。
- Remote: 检查 URL、headers、OAuth。

- 如果 Server 连上但 Tool 不可用，检查 permission/tool visibility。
- 如果模型不用，先把 Prompt 写清“什么时候需要用该 MCP”，并减少不相关 MCP。

8.7 一份最小 debug checklist

- [] 我从正确项目目录启动 OpenCode
- [] 文件放在正确目录且命名正确
- [] description 足够具体
- [] 没有同名覆盖/优先级问题
- [] permission 没有 deny
- [] 我能手动触发它
- [] 手动触发成功后，再测试自动路由
- [] MCP 的连接状态正常
- [] 没有一次启用过多 Tool/Skill 导致选择混乱

09

设计原则：什么时候该用谁？

从“文件格式记忆”升级成“系统设计判断”。

9.1 五问决策法

先问什么	如果答案是“是”	优先机制
这条信息几乎每个任务都要遵守吗？	是	AGENTS.md / instructions
我需要独立角色/权限/模型吗？	是	Agent
我想复用一套复杂做事方法吗？	是	Skill
我想让用户输入 /xxx 一键开始吗？	是	Command
我需要真实访问外部系统/工具吗？	是	MCP（或 Custom Tool）

9.2 “单一职责”是配置可维护性的核心

不要在 Agent Prompt 复制完整项目规范；不要在 AGENTS.md 复制每个 Skill；不要让 Command 重复 Skill 的 100 行流程；不要把 MCP 当成知识库说明书。每层只表达自己的职责。

9.3 低耦合组合方式

AGENTS.md：只说“什么时候查文档”
Skill：只说“怎么 review”
Agent：只说“reviewer 的角色 + 权限”
Command：只说“收集什么输入 + 交给谁”
MCP：只提供“查文档”的真实工具

9.4 初学者最常见的 8 个反模式

- 42. AGENTS.md 1000+ 行，把所有知识永久塞进 Context。
- 43. 为每个 Prompt 建一个 Agent，导致角色数量失控。
- 44. Skill description 全都叫“help with coding”，模型无法路由。
- 45. Command 中复制同一份长流程，维护时改一处漏三处。
- 46. 一次接入十几个 MCP，以为“工具越多越聪明”。
- 47. 所有权限都 allow，失去边界；或所有权限都 ask，产生 approval fatigue。
- 48. 同时使用 legacy tools 与新 permission，自己都说不清谁覆盖谁。
- 49. 没有最小验证，只看“模型回答看起来不错”，不检查实际触发链路。

9.5 推荐的成熟度路线

阶段	你应该有的东西	暂时不要做
Level 0	默认 Build/Plan + /init	自定义多 Agent
Level 1	精简 AGENTS.md	堆 MCP
Level 2	1-2 个只读专业 Subagent	复杂 orchestrator
Level 3	2-5 个高价值 Skill + 2-3 个 Command	几十个低质量 Skill
Level 4	按需 MCP + 精细 permission	全局启用所有 MCP
Level 5	评测、版本化、团队共享配置	只靠主观感觉优化

10

附录：可复制模板与学习路线

第一次照抄没问题；第二次要开始理解每个字段为什么存在。

10.1 最小 AGENTS.md 模板

```
# Project Name

## Structure
- `<path>`: <responsibility>

## Commands
- Install: `<command>`
- Focused test: `<command>`
- Full test: `<command>`
- Typecheck/lint: `<command>`

## Architecture
- <where business logic belongs>
- <existing abstractions to reuse>

## Working rules
- Make the smallest safe change.
- Verify the focused behavior first.
- Inspect the diff before finishing.

## Safety
- Do not push/deploy/change secrets without explicit request.
```

10.2 最小只读 Subagent 模板

```
---
description: <specific task and when to use this agent>
mode: subagent
temperature: 0.1
permission:
  edit: deny
  bash:
    "*": ask
    "git diff": allow
---

You are a read-only specialist.
Focus on <scope>.
Do not modify files.
Return evidence-backed findings.
```


10.3 最小 Skill 模板

```
---
name: my-skill
description: <specific trigger + specific outcome>
---

# Goal
<What this workflow achieves>

# When to use
<Trigger conditions>

# Procedure
1. ...
2. ...
3. ...

# Guardrails
- ...
- ...

# Output
- ...
```

10.4 最小 Command 模板

```
---
description: <what the command does>
agent: <optional-agent>
---

Task: $ARGUMENTS

Relevant context:
!`git diff`

Use <skill-name> when relevant.
Return <expected output>.
```

10.5 最小 Remote MCP 模板

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "docs": {
      "type": "remote",
      "url": "https://example.com/mcp",
      "headers": {
        "API_KEY": "{env:MY_MCP_API_KEY}"
      }
    }
  }
}
```

10.6 2 小时入门实操路线

时间	任务	你要理解的核心
0-20 min	运行 /init, 精简 AGENTS.md	常驻规则与项目记忆
20-45 min	做 reviewer Subagent	角色 + 权限边界
45-70 min	做 review-checklist Skill	按需加载 + description 路由
70-90 min	做 /review-changes	人工入口 + 参数/上下文注入
90-110 min	接一个只读 MCP	外部能力 + Context 成本
110-120 min	跑完整链路并排错	Discovery → Routing → Permission

10.7 7 天进阶路线

- 50. Day 1: 只优化 AGENTS.md, 观察不同任务是否稳定遵守。
- 51. Day 2: 熟练 Build / Plan / Explore, 再做 1 个 reviewer。
- 52. Day 3: 把最常重复的一套排错方法写成 Skill。
- 53. Day 4: 把最高频 2 个 Prompt 写成 Command。
- 54. Day 5: 接一个 MCP, 并测量启用前后的 Context/选择行为。
- 55. Day 6: 给 Agent 做最小权限设计, 练习 allow / ask / deny。
- 56. Day 7: 把一个真实任务跑 5 次, 记录成功率、工具误用、审批次数、上下文噪声, 再迭代配置。

10.8 最终自测：你真的分清五者了吗？

Q：“所有 TypeScript 改动最后必须 typecheck” 放哪里？

答案 AGENTS.md（高频、全局、稳定规则）

Q：“排查 flaky test 的 8 步方法” 放哪里？

答案 Skill（按需流程知识）

Q：“只读安全审计，不允许改文件” 怎么表达？

答案 Agent + permission

Q: “输入 /review 一键收集 git diff 并审查” 怎么做?

答案 Command

Q: “读取 Sentry issue / 查外部文档” 怎么做?

答案 MCP (真实外部能力)

Q: “reviewer 既要方法论又要只读边界” 怎么做?

答案 Agent + Skill 组合

10.9 来源与版本说明

本教程以 OpenCode 主线官方文档为事实基础，核验日期 2026-08-13。OpenCode 同时存在 /v2/ 预览/新版本文档，其部分配置结构与主线不同（例如 MCP 配置形态可能不同）。本教程为降低入门认知负担，统一使用主线 `opencode.ai/docs` 语法，不混用 v2 预览语法。

- Rules / AGENTS.md: <https://opencode.ai/docs/rules/>
- Agents: <https://opencode.ai/docs/agents/>
- Agent Skills: <https://opencode.ai/docs/skills>
- Commands: <https://opencode.ai/docs/commands/>
- MCP servers: <https://opencode.ai/docs/mcp-servers/>（站点可能重定向到 `dev.opencode.ai`）
- Permissions: <https://opencode.ai/docs/permissions>
- Config: <https://opencode.ai/docs/config/>

最后一句 不要追求“配置最多”。真正成熟的 OpenCode 项目，是每一层职责清楚、触发可解释、权限可验证、上下文不过载。